Algorithm 6: Pivot-Triggered One-Pass Removal (SOP-RLR)

Input: Graph $G(V, E)$, Current Step h, Current Labels $L_{\leq h}$
Output: Updated labeling with pruned redundancy $L_{\leq h}$

1 $r_{h-1} \leftarrow \max\{r(w) \mid (w, \cdot) \in L_{h-1}\}$, $r_h \leftarrow \max\{r(w) \mid (w, \cdot) \in L_h\}$ ($-\infty$ if $L_h = \emptyset$);
2 $W_h \leftarrow \{w \in V \mid r_h < r(w) \leq r_{h-1}\}$ ▷ Hubs finished after step h;
3 **if** $W_h = \emptyset$ **then return** $L_{\leq h}$;
4 $D[v] \leftarrow \emptyset,\ \forall v \in V$;
5 **foreach** $w \in W_h$ **do**
6 **foreach** $v \in V$ *with* $w \in C(v)$ ***in parallel*** **do**
7 **foreach** $(u, d) \in L[v]$ *with* $r(u) < r(w)$ ***and*** $w \in C(u)$ **do**
8 **if** $L[v][w] + L[u][w] \leq d$ **then**
9 $D[v].insert(u)$ ▷ Mark the entry for deletion
10 $L[v] \leftarrow \{(u, d) \in L[v] \mid u \notin D[v]\},\ \forall v \in V$;
11 **return** $L_{\leq h}$

A PROOFS

A.1 Proof of Theorem 1

Case 1: $L(u)[w] + L(v)[w] < L(v)[u]$. In this case, the common hub w provides a shorter connection between u and v than that represented by $L(v)[u]$. Hence, $L(v)[u]$ cannot contribute to any shortest distance query and is therefore redundant. **Case 2:** $L(u)[w] + L(v)[w] = L(v)[u]$. Since w lies on a path between u and v and has a higher order than u, vertex u cannot be the highest-order vertex among the vertices on that path. Consequently, $L(v)[u]$ must violate the canonical labeling constraint stated in Constraint 2, which requires u to be the highest-order vertex on every shortest path between u and v. Note that $r(w) > r(u)$ already implies $w \neq u$, and the condition holds as long as one such w exists, no matter how many shortest paths there are. Therefore, $(u, L(v)[u])$ is non-canonical, and thus redundant. Since $(u, L(v)[u])$ is redundant in both cases, the theorem follows.

A.2 Proof of Theorem 2

By Theorem 1, if there exists a vertex w satisfying $r(w) > r(u)$, $(w, L(u)[w]) \in L(u)$, $(w, L(v)[w]) \in L(v)$, and $L(u)[w]+L(v)[w] \leq L(v)[u]$, then the label $(u, L(v)[u]) \in L(v)$ is redundant. It remains to show that once every vertex with rank higher than $r(v)$ has finished propagation, the absence of such a vertex implies that $(u, L(v)[u])$ is non-redundant. Assume, for contradiction, that all vertices w satisfying $r(w) > r(v)$ have completed propagation, no vertex satisfies the condition of Theorem 1, yet $(u, L(v)[u])$ is still redundant.

We consider the two possible types of redundant labels. **Type-2 redundancy:** $L(v)[u] > dist(u, v)$. Since all higher-ranked vertices have completed propagation and the WPSL[68] index satisfies the 2-hop cover property, the query operation over $L(u)$ and $L(v)$ returns the exact shortest-path distance: $Q(L(u), L(v)) = dist(u, v)$. Therefore, $Q(L(u), L(v)) < L(v)[u]$. Let w be the common hub that realizes the query result. Then $L(u)[w] + L(v)[w] < L(v)[u]$. Thus, w satisfies the condition of Theorem 1, implying that $(u, L(v)[u])$ can be identified as redundant, which contradicts the assumption that no such vertex exists.

Type-3 redundancy: $L(v)[u] = dist(u, v)$, but $(u, L(v)[u])$ is non-canonical. Then there exists a vertex w on a shortest path between u and v such that $r(w) > r(u)$. Let w' be the highest-ranked vertex among all vertices appearing on any shortest path between u and v. Since the WPSL construction preserves all canonical labels, the entries $(w', L(u)[w']) \in L(u)$ and $(w', L(v)[w']) \in L(v)$ must both exist after propagation completes. Moreover, $L(u)[w'] + L(v)[w'] = dist(u, v) = L(v)[u]$. Hence, w' satisfies all conditions of Theorem 1. Therefore, $(u, L(v)[u])$ can be identified as redundant, again contradicting the assumption that no such vertex exists.

Both cases lead to contradictions. Therefore, after all vertices with rank higher than $r(v)$ have completed propagation, if no vertex satisfies the condition of Theorem 1, then $(u, L(v)[u])$ cannot be redundant. Equivalently, it must be canonical. Hence, the redundancy of labels in $L(v)$ can be completely verified once every vertex w satisfying $r(w) > r(v)$ has finished propagation.

A.3 Proof of Lemma 1

A label with hub w in L_h can only be generated from a label with hub w in L_{h-1} (line 5 of Algorithm 1). Hence, if no label of w is generated in L_{h-1}, no label of w is generated in L_h either, and by induction in any later step. The set of propagating hubs therefore shrinks monotonically, and so does its maximum rank r_h.

A.4 Canonicity of the Pivot-Triggered Removal (SOP-RLR)

Algorithm 6 gives the pivot-triggered removal described in Section 3.2; its check is in line 8. The rank intervals $(r_h, r_{h-1}]$ partition the ranks, so every hub is processed as a pivot in exactly one batch W_h, and by Lemma 1 its labels are final at that time. Consider a redundant label $(u, d) \in L(v)$. By Theorem 2, once all the vertices ranked above $r(v)$ have finished propagating, some hub w with $r(w) > r(u)$ satisfies the condition of Theorem 1 for (u, d), and w is processed in the batch of the step at which it finishes. If (u, d) already exists when w is processed, line 8 of Algorithm 6 marks it. If (u, d) is generated at a later step, the pre-insertion query $Q(L_{<h}[u], L_{<h}[v])$ of Algorithm 1 already returns a distance no larger than d through w, so the entry is never inserted. Hence, no Type-2 or Type-3 redundant label survives, while the Type-1 redundancy is removed by the insertion semantics. Conversely, every removal is certified by Theorem 1, and the witnesses of the removed labels are canonical labels that are never removed, so the 2-hop cover is preserved. Therefore, SOP-RLR outputs the canonical labeling.

A.5 Proof of Theorem 3

1) The PLL searches run from the top-k vertices in decreasing rank order, so they produce exactly the canonical labels whose hubs are among the top-k [7]. 2) The propagation generates every canonical label $(w, dist(w, v))$ with w outside the top-k: along a shortest path from w to v with the fewest hops, w is the highest-ranked vertex of every prefix, so by induction the candidate of each prefix is proposed and accepted unless it already exists, because a pre-insertion answer no larger than its distance would place a hub ranked above w on a shortest path. Hence, the converged labeling contains all the canonical labels and satisfies the 2-hop cover. 3) In the final scan, every hub has finished, so by Theorem 2 each non-canonical entry has a witness and is removed, whereas a canonical entry has none and survives. Since the witnesses are canonical labels that survive, the 2-hop cover is preserved and the output is exactly the canonical labeling.

A.6 Correctness of the Parallel Construction and Maintenance

The parallel propagation phases partition their output by vertex. In SWPSL, each propagation step reads only the finalized label sets L_{h-1} and $L_{<h}$, and each vertex writes to its own output set $L_h[v]$. Therefore, no concurrent read-write conflicts occur between threads, ensuring that parallel execution produces the same stepwise labels as sequential execution. The top-k PLL labels are canonical and remain available to every pruning query. Propagation preserves the 2-hop cover and the canonical entries. After convergence, SWPSL's final cleanup reads a fixed snapshot and writes survivors by vertex to a separate output. An entry is removed only by a shorter answer or an equal answer through a different pivot. Canonical entries cannot satisfy either condition, and their surviving witnesses certify all other removals. Thus, the parallel cleanup preserves the 2-hop cover and produces the canonical labeling. The direct SOP-RLR scheme is separate from this construction. Its pivot checks read a fixed snapshot within each batch and apply the deletion marks only after the batch.

In Algorithm 3, L, $L^{\downarrow}$, and $A^{\downarrow} = \{v \mid L^{\downarrow}[v] \neq \emptyset\}$ are fixed before filtering. Each phase reads these inputs and writes only to the output $L'[v]$ of its assigned vertex. The phases execute in sequence. No query reads an output being modified by another thread. Algorithm 2 shows LCP in push form. An equivalent parallel pull form assigns each target vertex to one thread, which gathers candidates from its neighbors' current frontiers, keeps the minimum candidate per hub, and publishes only strict improvements over the accumulated $L^{\downarrow}$ at the end of the round. This produces the same next frontier as the push form.

In Algorithm 4, the edge phase reads the original snapshot L_G, and the deletion requests of the increased edges that share an endpoint are collected independently and combined by set union at the synchronization point. During step-h validation, each vertex produces its own requests, which are published after that step. Predecessor checks read the deletion state committed by earlier phases, so marked predecessors cannot provide support. Physical deletion from L_G is deferred until detection finishes. Self labels are never marked. In Algorithm 5, the labeling L is read-only within a round, each thread only writes to its own $L_{(t)}[v]$, and $L_{(t)}$ is merged into L only after the round (line 11), which avoids any read-write conflict. Therefore, the parallel execution of all the algorithms generates exactly the same labels as the sequential one.

A.7 Proof of Theorem 4

We prove by contradiction. Assume there exists a vertex pair (u, v) such that after edge-weight decreases, the shortest distance in the updated graph $G^{\downarrow}$ satisfies $d_{G^{\downarrow}}(u, v) < Q(L[u] \cup L^{\downarrow}[u], L[v] \cup L^{\downarrow}[v])$ and the 2-hop cover constraint is violated. Let P be a shortest path from u to v in $G^{\downarrow}$. We consider two cases: **Case 1:** P contains no edge with decreased weight. Then $d_{G^{\downarrow}}(u, v) = d_G(u, v)$, and since the original label L_G satisfies the 2-hop cover constraint, we have $Q(L[u], L[v]) = d_G(u, v)$, which contradicts our assumption. **Case 2:** P contains at least one edge $e(a, b)$ whose weight has decreased. Since $\phi^{\downarrow}(a, b) < Q(L_G[a], L_G[b])$, the edge (a, b) is added to the active set $\mathcal{E}$ (lines 2-5). WCP phase ensures that any vertex whose distance to b (or a) becomes shorter through the decreased edge is captured in $L^{\downarrow}$. Furthermore, the LCP phase iteratively propagates these new labels to their neighbors. ensuring that all distance decreases are fully propagated throughout the graph. By induction on the propagation rounds, any vertex pair whose shortest distance decreases will eventually have a corresponding label in $L \cup L^{\downarrow}$ that correctly answers their distance via some hub. This contradicts our initial assumption that (u, v) violates the 2-hop cover constraint. Therefore, $L \cup L^{\downarrow}$ satisfies 2-hop cover constraint. The strict-improvement check against $L^{\downarrow}$ does not suppress a necessary propagation: a rejected candidate has an equal or shorter entry for the same hub already accumulated, whose propagation is either pending or has already occurred. Extending that entry over any edge yields a distance no larger than extending the rejected candidate. Thus omitting equal or worse candidates from the next frontier preserves the argument above.

A.8 Proof of Theorem 5

Let $M = L \cup L^{\downarrow}$, keeping the minimum distance for each hub. By Theorem 4, M answers every distance exactly. Its entries are upper bounds on the updated distances and respect the hub rank order. Consequently, M contains every canonical entry for this order: for a pair (u, v) for which u is highest-ranked over all shortest u–v paths, a covering pivot has rank at least $r(u)$, and a higher-ranked pivot would lie on a shortest u–v path, contradicting the canonical property. Hence the covering pivot must be u, and the exact entry is present at v.

No canonical entry (u, d) can satisfy the redundancy predicate. A query cannot return less than the true distance d. An equal answer through a different pivot would place a higher-ranked vertex on a shortest u–v path. Thus all canonical entries survive. Conversely, an overestimated entry has $q_g < d$, and a dominated entry has $q_g = d$ with an alternative higher-ranked pivot. The canonical entries witnessing these answers also survive, so simultaneous filtering does not introduce a circular dependency or lose coverage. In particular, an equal answer through u alone is not a deletion certificate, whereas a shorter new entry with hub u correctly eliminates its old longer copy.

The four local queries enumerate all old/new entry combinations, so their predicates have the same disjunction as the global redundancy test. For a new entry, the pre-insertion test gives $q_1 > d$. For an old entry, the original canonical labeling gives $q_1 = d$ and $P_1 = \{u\}$. Thus R_1 is false in both phases. Since $A^{\downarrow}$ is fixed from the unfiltered input $L^{\downarrow}$, every skipped local query has an empty input side and returns $(+\infty, \emptyset)$. The four cases therefore omit no possible deletion certificate, and Case 1 explicitly copies its old entry to L'. The output contains exactly the canonical entries and satisfies the 2-hop cover constraint.

A.9 Proof of Lemma 2

By the step invariant (Section 3.3), a label $(u, d) \in L_1[v]$ of step 1 is the edge label of $e(u, v)$ with $d = \phi_G(u, v) = dist_G(u, v)$. If $e(u, v) \notin E^{\uparrow}$, the edge still has length d in $G^{\uparrow}$, so $dist_{G^{\uparrow}}(u, v) \leq d$ and the label remains valid. Hence, only the edge labels of the increased edges can become invalid among the labels of step 1. Such an edge label $(b, \phi_G(a, b)) \in L_1[a]$ is invalid if and only if $dist_{G^{\uparrow}}(a, b) > \phi_G(a, b)$, *i.e.,* no other path of length $\phi_G(a, b)$ connects a and b. Algorithm 4 removes it in both cases, because checking the condition would

require a shortest path search per edge, and a valid edge label that survived with step 1 would no longer be an edge label, which breaks the step invariant for the later batches.

A.10 Proof of Lemma 3

Each label $(w, d) \in L_h[v]$ is generated from a predecessor $(w, d') \in L_{h-1}[u]$ via some neighbor $u \in N(v)$, where $d = \phi(v, u) + d'$. An increased connecting edge or an invalid predecessor can destroy this particular support. This alone does not imply that (w, d) is invalid, because another predecessor may still certify a path of length d. Conversely, if (w, d) is supported by at least one edge $e(v, u) \notin E^{\uparrow}$ and a valid predecessor $(w, d') \in L_{h-1}[u]$, then the path $\langle v, u, \ldots, w\rangle$ still has length d in $G^{\uparrow}$, so $d \geq dist_{G^{\uparrow}}(v, w)$ and (w, d) cannot underestimate the updated distance, *i.e.,* it is valid.

A.11 Proof of Theorem 6

All step-0 self labels remain valid and are retained. We prove removal completeness by induction on the positive generation steps. A label whose hub is in $D[v]$ is treated as absent by *CheckPredecessor*, and all marks from earlier phases are visible before the next step begins. **[Base]:** *All invalid labels at step $h = 1$ are removed.* Suppose $(u, d) \in L_{G,h=1}[v]$ is an invalid label. Since it has step 1, it is an edge label (Section 5), so $d = dist_G(u, v) = \phi_G(u, v)$, meaning that the shortest path $sp_G(u, v)$ is the edge $e(u, v)$. If (u, d) is invalid, then $dist_{G^{\uparrow}}(u, v) > dist_G(u, v) = \phi_G(u, v)$, so $e(u, v)$ has increased (Lemma 2), and the edge phase of Algorithm 4 marks (u, d) at v while retaining the self label $(u, 0)$ at u. Hence every invalid step-1 entry is marked without deleting a self label. **[Induction]:** *All invalid labels at step $h > 1$ are removed, provided that all invalid labels at step $h - 1$ have been removed.* Let $(w, d) \in L_{G,h}[v]$ be invalid, *i.e.,* $d < dist_{G^{\uparrow}}(v, w)$, and consider any neighbor $x \in N(v)$ examined by *CheckPredecessor*, *i.e.,* $e(v, x) \notin E^{\uparrow}$, with a predecessor $(w, d') \in L_{G,h-1}[x]$ such that $d' = d - \phi(v, x)$. Since $\phi^{\uparrow}(v, x) = \phi(v, x)$, the triangle inequality gives $dist_{G^{\uparrow}}(x, w) \geq dist_{G^{\uparrow}}(v, w) - \phi(v, x) > d - \phi(v, x) = d'$, so the predecessor (w, d') is itself invalid and has been marked by the induction hypothesis, hence $w \in D[x]$. The neighbors with $e(v, x) \in E^{\uparrow}$ are skipped by *CheckPredecessor*. Hence, (w, d) has no valid predecessor left and is marked at step h. Conversely, a label that keeps a valid predecessor is valid by Lemma 3, so the removal never stops before all the invalid labels are found, and Algorithm 4 removes all of them.

Therefore, the completeness of Algorithm 4 relies only on the base step: once the edge labels of the increased edges are removed, every invalid label of a higher step is found by the predecessor validation, which is why the edge phase needs neither a distance check nor the pivots between the endpoints.

Remark. Lemma 3 gives a sufficient, but not necessary, condition for retaining a label. Thus, *CheckPredecessor* may conservatively remove a valid label when all its recorded predecessor supports from step $h - 1$ have been invalidated or removed, even if an equal-length path remains through a label generated at a different step, and the edge phase likewise removes the edge label of an increased edge even when another path of the same length keeps it valid. This removal is safe: Theorem 6 guarantees that no invalid label survives, and the recovery phase in Section 5.2 restores the 2-hop cover by generating any labels needed for coverage. A removed label need not be reinserted if the remaining labels already cover the corresponding vertex pair.

A.12 Proof of Pruning Condition 1

If neither v nor w belongs to $\mathcal{A}^{\uparrow}$, then neither of them loses any label in the deletion phase (Algorithm 4). Hence, their original label sets $L_G[v]$ and $L_G[w]$ remain valid and can answer $Q(L_G[v], L_G[w]) = dist_G(v, w)$ correctly. Since the increased edge cannot shorten any shortest path, we have $dist_G(v, w) \leq dist_{G^{\uparrow}}(v, w)$. Therefore, the existing valid labels already provide the correct distance in $G^{\uparrow}$, that is, $Q(L[v], L[w]) = dist_{G^{\uparrow}}(v, w)$. Then inserting a label (w, d) into $L(v)$ would be redundant, as $dist(v, w)$ is already covered by the existing hubs. Consequently, such vertex pairs can be safely pruned during propagation.

A.13 Proof of Pruning Condition 2

Since $w \notin \mathcal{A}^{\uparrow}$, the label set $L_G[w]$ remains unchanged and thus valid in $G^{\uparrow}$. The pivot vertex x that witnesses a shortest distance between two vertices (*i.e.,* $Q(L_G[u], L_G[v])$) satisfies $r(x) \geq \max\{r(u), r(v)\}$. If the maximum deleted rank at v is $Max_r(v) < r(w)$, then every pivot x with $r(x) > r(w)$ in $L_G[v]$ survives the deletion phase. Consequently, the distance query using the *remaining* labels at v and the unchanged labels at w still attains the updated distance: $Q(L[v], L[w]) = dist_{G^{\uparrow}}(v, w)$. Therefore, exactly as in Pruning Condition 1, the candidate (w, d) is already covered by the existing hubs and can be safely skipped without further testing.

A.14 Proof of Theorem 7

If the pruning conditions (line 7) are disabled, Algorithm 5 is equivalent to the standard WPSL construction, which has been proven to guarantee complete 2-hop coverage, so the unpruned version is correct. The two pruning conditions only exclude label pairs that are already correctly covered by existing valid hubs, so they merely prune the redundant candidates, without affecting the correctness. Hence, the resulting labeling $L_{\text{old}} \cup L_{\text{new}}$ preserves the 2-hop cover property and remains fully correct under $G^{\uparrow}$.

A.15 Proof of Theorem 8

As all the invalid labels are deleted from the original canonical G's label set G, the remaining labels L_{old} are still valid and canonical in G. Hence, it cannot contain *Type-1* or *Type-2* redundancy. The only possible redundancy would be *Type-3*, where a label $(u, d) \in L_{old}[v]$ could be dominated by another vertex x if $r(x) > r(v)$, $r(x) > r(u)$, and $dist_{G^{\uparrow}}(v, x) + dist_{G^{\uparrow}}(x, u) = dist_{G^{\uparrow}}(v, u) = d = \text{dist}_G(v, u)$. However, since $dist_{G^{\uparrow}}(v, x) \geq dist_G(v, x)$ and $dist_{G^{\uparrow}}(x, u) \geq dist_G(x, u)$, such a vertex x must have also dominated (u, d) in the original graph G. Therefore, (u, d) would never appear in $L_G[v]$ under a canonical labeling scheme. This contradiction proves that no label in L_{old} can become dominated under $G^{\uparrow}$. Hence, L_{old} is free of redundancy.

B COMPLEXITY ANALYSIS

We follow the notation of Section 3.3, where $n = |V|$, $m = |E|$, p is the number of threads, δ is the maximum label size of a vertex, and a distance query takes $O(\delta)$ time.

B.1 Index Construction (Section 3.3)

Propagation depth versus topological diameter. The unweighted hop distance minimizes the number of edges, whereas a weighted shortest path minimizes the total edge weight. These quantities need not coincide: consider a complete graph on $n \geq 3$ vertices, with weight 1 on the edges of a spanning path and weight n on all other edges. Its unweighted diameter is 1, but the weighted shortest path between the spanning path's endpoints contains $n-1$ edges. Thus, small topological diameter alone does not guarantee few-hop weighted shortest paths. We use the actual number of propagation rounds H in the analysis below, rather than identifying H with the unweighted hop diameter.

Hybrid construction. For sequential PLL initialization, the k graph searches cost $O(k(m + n \log n))$. The i-th search performs at most n pruning queries over at most $O(i)$ existing hubs per label set. These queries add $O(nk^2)$ work across all searches. In each propagation round, a vertex v collects $O(D(v)\delta)$ candidates and checks each with an $O(\delta)$-time query, giving $O(Hm\delta^2)$ total work. After propagation converges, the final cleanup scans at most $O(n\delta)$ entries in a fixed labeling. Each entry requires one query returning the minimum distance and all minimizing pivots, which takes $O(\delta)$ time. Thus, this final scan costs $O(n\delta^2)$ work. This bound does not rely on the per-pivot one-pass property of SOP-RLR. The total work is $O(k(m+n\log n)+nk^2+Hm\delta^2+n\delta^2)$, with $O(n\delta)$ space for labels and temporary outputs. Under ideal load balance and ignoring synchronization overhead, distributing propagation and cleanup over p threads gives $O(k(m + n\log n) + nk^2 + (Hm\delta^2 + n\delta^2)/p)$ time.

Direct SOP-RLR. Let δ_S be the maximum label size during SOP-RLR's execution. Each pivot is processed once, but scans all n vertices and up to δ_S entries at each eligible vertex. With constant-time hub lookups, the pruning loops of Algorithm 6 therefore require $O(n^2\delta_S)$ work in the worst case. A label can be checked against multiple pivots. This pruning cost is not incurred by the hybrid construction.

B.2 Decrease Maintenance (Section 4)

Let H_d be the number of LCP rounds and m_d be the maximum sum of degrees of the vertices that receive new labels in one round. In Algorithm 2, the initial detection takes one query per decreased edge, *i.e.,* $O(|E^{\downarrow}|\delta)$ time. WCP checks $O(\delta)$ candidate hubs for each active edge with one query each, so it takes $O(|E^{\downarrow}|\delta^2)$ time, and LCP takes $O(H_d m_d \delta^2)$ time, since each vertex with new labels propagates $O(\delta)$ labels to each of its neighbors with one query per candidate. Therefore, Algorithm 2 takes $O(|E^{\downarrow}|\delta^2 + H_d m_d \delta^2)$ time in total. In Algorithm 3, the new labels, at most $O(H_d m_d \delta)$ in total, are each checked with three queries in $O(|L^{\downarrow}|\delta)$ time, while the old labels are only checked for the pairs with an affected endpoint, resulting in a worst-case bound of $O(n\delta^2)$, as pairs with no affected endpoints (Case 1) are skipped without any query. Both algorithms are parallelized over vertices with p threads (Section 4.2), and the extra space is the temporary new label set $L^{\downarrow}$, which is released after the update.

B.3 Increase Maintenance (Section 5)

The edge phase of Algorithm 4 removes the edge label of each increased edge with a constant-time lookup per edge, so it takes $O(|E^{\uparrow}|)$ time. The predecessor validation checks each label against the labels of its neighbors, so each step takes $O(\sum_v D(v)\delta) = O(m\delta)$ time and all the steps take $O(Hm\delta)$ time in total, where $H = maxStep$ is the maximum recorded generation step in the input index. Let H_i be the number of recovery rounds and m_i be the maximum sum of degrees of the vertices that receive new labels in one round. The pruned propagation in Algorithm 5 takes $O(H_i m_i \delta^2)$ time, since each surviving candidate incurs one query, while the two pruning conditions skip the candidates of the unaffected pairs without any query. The redundancy removal on the recovered labels takes $O(|L_{new}|\delta)$ time with $|L_{new}| = O(H_i m_i \delta)$. The edge phase is parallelized over the updated edges, while predecessor validation, recovery, and redundancy filtering are parallelized over vertices with p threads, and the extra space is the one-byte generation step of each label together with the temporary L_{new}, which is released after the update. Compared with the PPR-based maintenance, SWPSL trades the dependency lookups of the PPR for the predecessor validation and the pruned propagation, both of which are bounded by the labels of the affected vertices and require no persistent structure.

C TOPOLOGY UPDATES

The edge and vertex insertions are reduced to the weight decrease in Section 4, and the edge and vertex deletions are reduced to the weight increase in Section 5, as detailed below.

C.1 Insertion Update Maintenance

We next discuss how to support *edge* and *vertex insertion* in SWPSL. Both operations can be reduced to the *edge decrease*, so they can be handled by the same mechanisms introduced in Alg. 2 and 3.

Edge insertion. Inserting a new edge $e(a, b)$ with weight w into graph G can be viewed as a special case of edge weight decrease. Specifically, we treat the original graph G as containing $e(a, b)$ with an *infinite* weight, which is then decreased to w. Under this formulation, the edge insertion is equivalent to adding $e(a, b)$ into the decreased edge set $E^{\downarrow}$. As a result, we can directly invoke the weight-decrease update procedures, including the new label propagation and the redundant label removal.

Vertex insertion. Inserting an isolated vertex alone does not affect the shortest distances and has no impact on the labels. We therefore consider the vertex insertion together with the insertion of its incident edges. When a new vertex v is inserted, we first initialize its label set as $L(v) = \{(v, 0)\}$. To preserve the canonical labeling order, the new vertex is assigned the new lowest rank. After that, each newly inserted incident edge $e(a, b)$ is processed as an edge insertion, and hence reduced to the edge weight decrease case described above.

C.2 Deletion Update Maintenance

This section discusses how to support *edge* and *vertex deletion*. Similar to the insertion, both operations can be reduced to the edge *increase* and handled by the increase update methods. **Edge deletion:** Deleting an edge $e(a, b)$ can be equivalently modeled as increasing its weight to infinity, so the removal of $e(a, b)$ is treated as an edge weight increase update. Therefore, we insert $e(a, b)$ into

Table 2: Construction time (s) under different distance units on BK and YF.

Dataset	Algorithm	Distance unit		
		10^4 m	10^3 m	10^2 m
BK	M2HL	10.18	107.02	1130
	SWPSL	**0.71**	**0.94**	**1.02**
YF	M2HL	1.77	14.81	130.34
	SWPSL	**0.15**	**0.17**	**0.27**

the increased edge set $E^{\uparrow}$ and directly apply the procedures for the weight increase, including the invalid label detection (Algorithm 4) and the subsequent label recovery (Algorithm 5). **Vertex deletion:** Deleting a vertex v requires deleting all the edges incident to v, so the vertex deletion is reduced to a set of edge deletions.

C.3 Memory during Maintenance

During maintenance, SWPSL uses temporary label sets, including the new label set $L^{\downarrow}$ for decreases (Algorithm 2) and the recovered label set L_{new} for increases (Algorithm 5), and these sets are released after the corresponding batch is processed. We report the three largest temporary label footprints observed during maintenance. For 1k decreases, the peak sizes of the temporary structures are 164.51MB, 344.59MB, and 707.58MB on BK, GO, and IC, respectively, which are comparable to the sizes of their canonical labels (Table 1-(c)), and for 1k increases, the recovered labels only take 3.15MB, 2.56MB, and 1.21MB. In contrast, the PPR of BPCL and WPSL has to be kept permanently and grows with the updates, and the position array of M2HL has to be kept for every label.

C.4 Influence of PLL Number k

This subsection gives the detailed analysis of Figure 9, which is summarized in Section 6.4.

As shown in Figure 9, we study the influence of the PLL number k in the hybrid construction on the indexing time and the size of the redundant label entries by varying it from 0 (WPSL), 1 to 500. The difference between the neighboring bars shows how many redundant labels can be pruned by the additional PLLs. First, the benefit of PLLs on the redundancy reduction is dramatic and immediate. For some networks, even a single PLL (k=1) drastically cuts the redundancy by 86.4% for WT (from 1,586MB to 216MB) and 84.5% for HO (from 30,037MB to 4,662MB). This redundancy reduction continues to improve with a larger k, but the marginal benefit diminishes after 200. Meanwhile, as PLL is slower than WPSL, the more PLLs we run, the longer it takes. Therefore, the construction time exhibits a clear U-shaped pattern with the optimal k falling around 1-50 for the smaller graphs and 100-200 for the larger graphs. As k further increases, the initial PLL cost begins to dominate, causing a gradual time increase. When k grows from 100 to 500, the construction time increases and slowly surpasses that of k=1.

Selection of k. Since the index size and the query time are invariant to k (Section 3.3), the selection only concerns the construction time. The experiments show that the optimum falls within $[10, 100]$ on 12 of the 13 graphs, and even on IC, where the optimum lies outside this range, the best k in $\{10, 50, 100\}$ is only 0.65s slower than the optimum, which is negligible compared with the gap to the baselines. Therefore, the rule of testing $\{10, 50, 100\}$ on a new graph and keeping the fastest one is sufficient in practice. Even a suboptimal k in this range is still much faster than all the baselines.

C.5 Sensitivity Studies

This subsection reports the detailed results of the sensitivity studies summarized in Section 6.5.

Thread Scalability. Figure 10 shows the construction time and the maintenance time of 1k updates under 8, 16, 32, and 64 threads on SK, WI, HO, BK, GO, and IC. All the phases of SWPSL scale with the thread number: increasing the thread count from 8 to 64 reduces construction, decrease maintenance, and increase maintenance times by factors of up to 5.04×, 5.00×, and 4.29×, respectively, all on IC. The scaling is sublinear and varies by workload. These consistent runtime reductions show that additional threads benefit SWPSL across both construction and maintenance workloads.

Update Magnitude. Figure 11 presents the maintenance time for the same batches of 1,000 edge weight updates while varying the multiplicative weight-scaling factor to study the impact of the change magnitude. For decreases, a smaller scaling factor represents a larger weight reduction, which may shorten more shortest paths and trigger additional label generation. The observed growth in maintenance time remains moderate: for example, reducing the factor from 0.8 to 0.2 increases the time from 3.25 s to 5.77 s on BK and from 16.55 s to 27.93 s on IC. For increases, maintenance time changes only slightly across the tested factors, ranging from 1.14 s to 1.28 s on GO and from 5.95 s to 6.10 s on IC. Overall, these results indicate that SWPSL maintains stable update performance across the evaluated weight-change magnitudes, with moderate sensitivity to decreases and limited sensitivity to increases.

Weight Granularity. To further examine the sensitivity to the numeric distance scale discussed in Section 1, Table 2 reports the construction time of M2HL and SWPSL on BK and YF under three different distance units, ranging from 10^4 m to 10^2 m. As the distance unit becomes finer, the construction time of M2HL increases dramatically on both datasets. Specifically, it increases from 10.18s to 1,130s on BK and from 1.77s to 130.34s on YF. In contrast, the construction time of SWPSL remains relatively stable on both datasets. These results indicate that SWPSL is substantially less sensitive to the granularity of distance weights than M2HL. Although M2HL is relatively efficient under coarse distance units, its construction cost grows rapidly as the distance representation becomes more fine-grained. This observation further demonstrates the robustness and scalability of SWPSL under different distance scales.

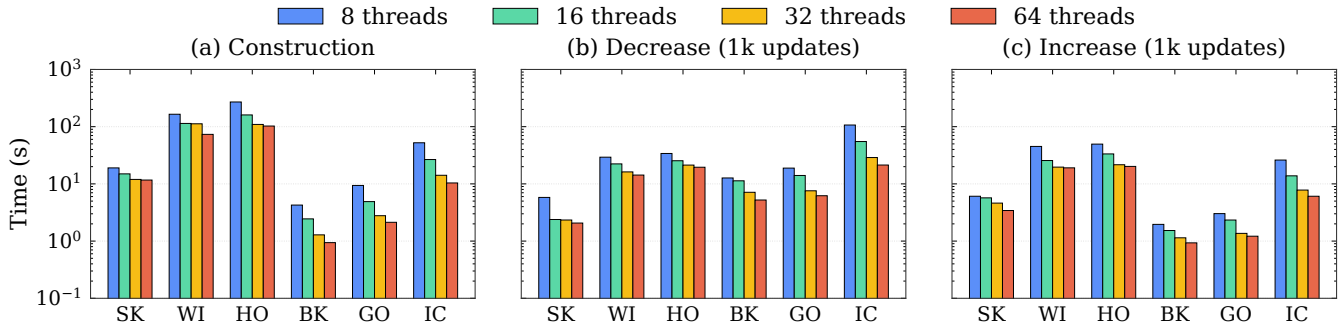


Figure 10: SWPSL Construction and Maintenance under Different Thread Counts

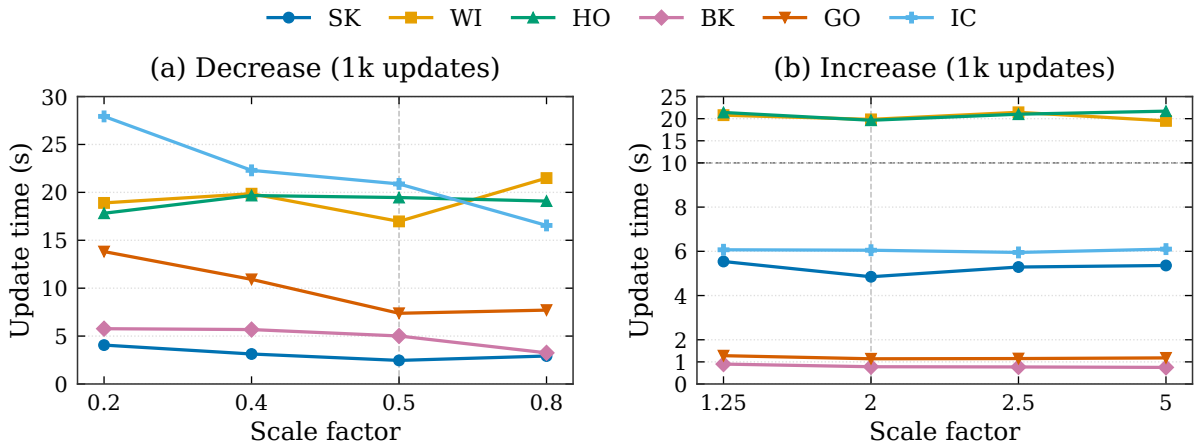


Figure 11: Maintenance of Different Scale Factors